



Othello/Reversi as a Reinforcement Learning Project

Othello, also known as Reversi, is a two-player strategy board game based on control, positioning, and long-term planning. In the standard version, the game is played on an 8×8 board. However, for this assignment, we will use a simplified 5×5 board in order to make the problem more manageable while preserving the main strategic and computational challenges of the game.

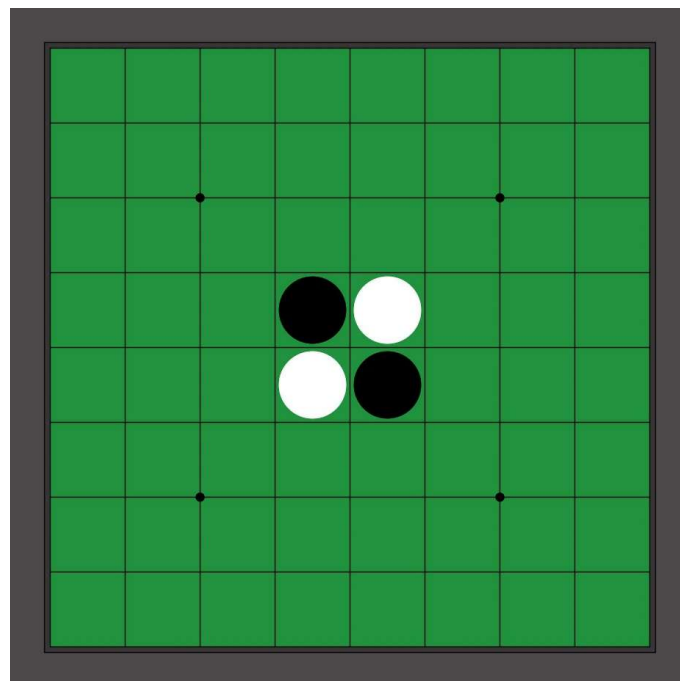
The two players are represented by black and white discs. Players take turns placing one disc of their color on the board. A move is legal only if it captures one or more opponent discs. Capturing occurs when opponent discs are trapped in a straight line — horizontally, vertically, or diagonally — between the newly placed disc and another disc of the current player's color. All captured discs are then flipped.

The objective of the game is to finish with more discs of your color than your opponent. The game ends when neither player can make a legal move.

Why Othello is Interesting for AI

Othello is a perfect-information game: both players can see the full board at all times, and there is no hidden information. However, despite its simple rules, the game has a very large number of possible board configurations. This makes it an excellent case study for reinforcement learning and Deep Reinforcement Learning.

The complexity grows very quickly with board size:





Even a 5×5 board has an enormous state space. Therefore, exhaustive search or tabular Q-learning becomes impractical. Instead of storing every possible state-action pair in a table, Deep Q-Learning uses a neural network to approximate the value of actions and generalize across similar board positions.

Deep Q-Learning and self-play are promising approaches because they allow the agent to:

- approximate strong strategies without explicitly solving the entire game;
- learn board patterns and generalize across similar positions;
- explore unusual states using epsilon-greedy exploration;
- improve through repeated interaction with the environment.

Board size	Cells	Possible states
4×4	16	43,046
5×5	25	8.4×10^{11}
6×6	36	7.9×10^{17}
8×8	64	$\sim 10^{30}$

Possible Agent Strategies

1. Random Agent

The Random Agent chooses one legal move at random. It does not learn and does not use any strategy. Its purpose is to provide a baseline. If a trained agent cannot consistently beat a random agent, then the learning process has probably failed.

2. Greedy / Heuristic Agent

The Greedy Agent selects the move that maximizes an immediate heuristic, such as the number of opponent discs flipped in the current turn. A more advanced heuristic may also value corners, edges, or stable positions.

This agent is simple but useful. It helps students compare learned strategies against a rule-based approach. However, greedy strategies may be short-sighted because flipping many discs immediately is not always the best long-term decision.



3. Deep Q-Learning Agent

The Deep Q-Learning Agent receives the board state as input and predicts Q-values for the possible actions. For a 5×5 board, the output layer can contain 25 values, one for each board position. Illegal moves must be masked or penalized.

The agent learns using:

- a neural network to approximate Q-values;
- epsilon-greedy exploration;
- replay buffer to stabilize training;
- target network to reduce instability;
- rewards based on winning, losing, drawing, or final disc difference.

The goal is to learn a policy that maximizes the cumulative reward over time.

4. Deep Q-Learning with Prioritized Experience Replay

This strategy extends standard Deep Q-Learning by giving more importance to transitions with larger temporal-difference errors. These are usually the transitions from which the agent can learn the most.

The purpose is to speed up learning and focus training on the most informative experiences.

5. Self-Play Agent

In self-play, the agent trains by playing against itself or against previous versions of itself. This can lead to stronger and more general strategies because the agent is not overfitted to a single opponent.

The agent may alternate between playing as black and white, learning from both perspectives.

6. Hybrid / Guided Agent

A hybrid agent combines Deep Q-Learning with simple heuristics. For example, the agent may be encouraged to prioritize corners, avoid dangerous positions, or value mobility.

This approach can help guide the learning process and may improve performance, especially in early training.



7. Multi-Agent Training

In this approach, different groups or different versions of the agent are trained with different hyperparameters, architectures, or reward functions. These agents can then play against each other.

This allows comparison of learning efficiency, stability, and final performance.

Objective of the Assignment

The objective of this assignment is to build the necessary tools to create, train, evaluate, and test an Othello-playing agent.

Students must:

- implement the Othello environment;
- define legal moves and board transitions correctly;
- develop an agent capable of playing the game;
- train the agent using Deep Q-Learning or another reinforcement learning approach;
- save the trained model to disk;
- load the trained model for evaluation;
- compare their strategy against other agents.

On the competition day, the instructor will organize an all-against-all tournament among the agents that pass the minimum requirements.

Minimum Requirements

To be eligible for the competition, the project must satisfy the following conditions:

- the program must run correctly without errors;
- the game rules must be correctly implemented;
- the agent must be trained and functional;
- the trained model must be saved and loaded from disk;
- the students must submit a written report of maximum 10 pages explaining their approach.

Meeting these requirements can provide up to **8 points**.

Competition Scoring

Agents that pass the minimum requirements will participate in an all-against-all tournament.



The competition score will be assigned as follows:

- first place: **2 points**;
- last place: **0.25 points**;
- intermediate positions: linear interpolation between **0.25 and 2 points**.

The final grade will be:

Final Score=Minimum Requirements Score + Competition Score

Group Work

The assignment can be completed individually or in pairs. Teams are encouraged to divide the work between environment implementation, agent design, training, evaluation, and documentation.

The project evaluates not only the final performance of the agent, but also the correctness of the implementation, the quality of the training process, and the clarity of the analysis.

Final Submission Requirement

The final submitted agent must not train again during the evaluation. It must only load the trained policy or model and choose actions during a game.

Each submission must include a Python class called:

```
class StudentAgent:
```

This class must contain the following methods:

```
def __init__(self, board_size: int, checkpoint_path: str | None =  
None):  
    pass  
def encode_state(self, observation: dict):  
    pass  
def select_action(self, observation: dict) -> int:
```

Board Representation

The game board is represented as a matrix with the following values:

```
1 = pieces belonging to the current agent  
-1 = pieces belonging to the opponent  
0 = empty cells
```



This means that the board is always given from the point of view of the player who has to move.

For example:

```
observation["board"]
```

contains the relative board seen by the agent. The absolute board is also available:

```
observation["board_abs"]
```

At each turn, the agent receives an observation dictionary with the following fields:

```
observation = {  
    "board": board,  
    "board_abs": board_abs,  
    "current_player": current_player,  
    "legal_actions": legal_actions,  
    "pass_action": pass_action,  
    "board_size": board_size,  
    "turn_count": turn_count  
}
```

The most important is `legal_actions`, containing the list of valid actions that the agent is allowed to choose.

For a board of size $N \times N$, the possible actions are: 0, 1, ..., $N*N-1$. Each number corresponds to a one position on the board, where

```
row = action // board_size  
col = action % board_size
```

There is also one special action: if the number is $N*N$, this action means that the player passes the turn. It is only legal when there are no valid moves available.

The agent must always return a legal action. That is, the output of:
`agent.select_action(observation)` must belong to:
`observation["legal_actions"]`

If an agent returns an illegal action, it will automatically lose the game.

Minimal Valid Agent Example

The following agent is valid, although it is not intelligent. It simply chooses the first legal action:



```
class StudentAgent:
    def __init__(self, board_size: int, checkpoint_path: str | None =
None):
        self.board_size = board_size
        self.checkpoint_path = checkpoint_path

    def select_action(self, observation: dict) -> int:
        legal_actions = observation["legal_actions"]
        return legal_actions[0]
```

Important Notes

- The training method is free, but the final interface is fixed.
- The submitted agent must be compatible with the following use:

```
agent = StudentAgent(board_size=6, checkpoint_path="model_file")

action = agent.select_action(observation)
```

- The agent must not require manual input during evaluation.
- The agent must not print excessive information during evaluation.
- The agent must not modify the rules of the game.
- The agent must not train again during the tournament phase.